

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

3ra. práctica (tipo B)
(Segundo Semestre 2025)

Duración: 1h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías `iostream`, `iostream`, `limits`, `cstring`, `cmath` o `fstream`
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como compilador CLion.**
- Los archivos deben llevar como nombre su código de la siguiente forma `codigo_LAB2_P#&` (donde # representa el número de la pregunta a resolver y & la alternativa)

Elija una de las 2 preguntas:

Pregunta 1 (20 puntos)

Un sistema de comercio electrónico necesita procesar **transacciones** de clientes de manera eficiente. Las transacciones pueden ser de tres tipos: PEDIDO: prioridad 1 (**más alta**), CANCELACIÓN: prioridad 2, CONSULTA: prioridad 3 (**más baja**), cada una con diferente nivel de urgencia.

Se declaran los siguientes datos:

Nombre de la variable	Tipo de Dato	Datos que maneja
transacción	ElementoColaPrioridad	- id : int, identificador de la solicitud - tipo : char[12], ("PEDIDO", "CANCELACIÓN" ó "CONSULTA") - prioridad : int

		- exitosa , booleano, estado de éxito.
mensaje	ElementoCola	- id : int, identificador de solicitud - mensajeParte1 : char[30], parte inicial de mensaje - mensajeParte2 : char[30], parte final de mensaje

Implemente el sistema descrito usando **cola de prioridad** para **transacciones** y **cola normal** para **mensajes**, demostrando el procesamiento por prioridades. El sistema debe demostrar que los PEDIDOS se procesan inmediatamente, incluso si llegaron después de otras transacciones, mostrando los mensajes correspondientes al usuario según si la transacción fue exitosa o falló.

Pasos Por Desarrollar

Pruebas iniciales (sin puntaje)

Compile y ejecute el proyecto. Observe que la salida de la función `probarColaPrioridad()`; funciona para dos prioridades.

Modificar la cola de Prioridad (6 puntos)

Adaptar la función `insertarConPrioridad()` existente de cola de prioridad para manejar **tres niveles de prioridad**. Estudie qué cambios son necesarios para manejar: PEDIDO, CANCELACIÓN y CONSULTA. La inserción debe ser $O(1)$. Para probar que se está encolando con prioridad correctamente use la función `probarColaPrioridad()`.

Proceso de Simulación

Implementar una simulación para un tiempo de 12 minutos.

Llegada de transacciones (4 puntos)

La llegada de las transacciones tiene una probabilidad del 90%. Use la función `int aleatorio(int desde, int hasta)`. Observe su uso dentro de la función `probarColaPrioridad()`.

Las nuevas transacciones llegan en forma aleatoria (es decir, los datos no se necesitan leer), use la función `aleatorio(desde, hasta)` para generar la llegada de la transacción con tres niveles de prioridad. Vuelva a usar la función `aleatorio(desde, hasta)` para hallar la probabilidad de éxito de la transacción, 70% de éxito. Guarde las transacciones.

Se muestra el mensaje: Nueva solicitud #1 PEDIDO (Prioridad: 1)

Dónde: 1, es el **id**; PEDIDO, es el **tipo**; y, 1 es la **prioridad**

Procesar transacciones (6 puntos)

Las transacciones se procesan cada 3 minutos.

Tomar en cuenta los siguientes mensajes que se deben generar. Estos se deben encolar en la **cola de mensajes**.

Se muestra el mensaje: Procesando PEDIDO #1

Dónde: PEDIDO, es el tipo; 1 es el id

Transacción	Tipo	Mensaje (mensajeParte1 + id + mensajeParte2)
Exitosa	Pedido	Pedido #1 confirmado
	Cancelación	Cancelación #2 procesada
	Consulta	Consulta #3 respondida
No Exitosa	Pedido	Error en pedido #1 - stock agotado
	Cancelación	Error en cancelación #2 - no permitida
	Consulta	Error en consulta #3 - sistema ocupado

Procesar mensajes (2 puntos)

Se envía un mensaje al usuario y muestra ESTADO cada 4 minutos.

Si la cola no está vacía se muestra el mensaje correspondiente. Por ejemplo:

Error en pedido #1 - stock agotado, caso contrario se muestra el mensaje: “No hay transacciones pendientes”.

Se muestra también el ESTADO de la cantidad de transacciones y mensajes. Por ejemplo:

ESTADO: 4 transacciones en cola de prioridad, 0 mensajes pendientes

Dónde: 4, es la longitud de la cola de transacciones y 0 es la longitud de la cola de mensajes.

Mostrar resumen Final (2 puntos)

Se muestran las transacciones procesadas, transacciones pendientes y los mensajes por enviar.

Salida esperada:

```
=====
SISTEMA DE PEDIDOS WEB (CON PRIORIDAD)
=====

Nueva solicitud #1 PEDIDO (Prioridad: 1)
Nueva solicitud #2 CONSULTA (Prioridad: 3)
Nueva solicitud #3 CANCELACION (Prioridad: 2)
```

Procesando PEDIDO #1

Nueva solicitud #4 PEDIDO (Prioridad: 1)

Pedido #1 confirmado

ESTADO: 3 transacciones en cola de prioridad, 0 mensajes pendientes

Nueva solicitud #5 PEDIDO (Prioridad: 1)

Nueva solicitud #6 CONSULTA (Prioridad: 3)

Procesando CANCELACION #3

Nueva solicitud #7 PEDIDO (Prioridad: 1)

Cancelación #3 procesada

ESTADO: 5 transacciones en cola de prioridad, 0 mensajes pendientes

Nueva solicitud #8 PEDIDO (Prioridad: 1)

Procesando CONSULTA #2

Nueva solicitud #9 PEDIDO (Prioridad: 1)

Nueva solicitud #10 CANCELACION (Prioridad: 2)

Nueva solicitud #11 PEDIDO (Prioridad: 1)

Procesando CONSULTA #6

Error en consulta #2 - sistema ocupado

ESTADO: 7 transacciones en cola de prioridad, 1 mensajes pendientes

RESUMEN FINAL

=====

Transacciones procesadas: 4

Transacciones pendientes: 7

Mensajes por enviar: 1

Pregunta 2 (20 puntos)

En empresa de producción tiene líneas con productos uno tras otro, semejante a una cola, cuando los productos entran a la cola deben estar debidamente codificadas con números consecutivos a partir del 1, el detalle es que al parecer los robots que colocan los productos en las líneas están fallando y los han colocado de forma inadecuada. Se necesita un programa que verifique si los productos de la cola se pueden colocar en otra cola ordenados empleando una **pila auxiliar** (no es necesario colocarlos en la segunda cola, solo verificar si se pueden encolar ordenados). A continuación, algunos ejemplos:

Caso 1:

7, 6, 5, 4, 1, 2, 3

Si se puede encolar ordenado

Caso 2:

4, 5, 6, 7, 1, 2, 3

No se puede encolar ordenado

Caso 3:

1, 5, 6, 2, 4, 7, 3

No se puede encolar ordenado

Caso 4:

1, 2, 3, 5, 4

Si se puede encolar ordenado

Desarrollar un algoritmo que **solo verifique**, si empleando una pila auxiliar puede colocar los productos ordenados en otra cola, para esta labor el desarrollo debe ser iterativo y debe tener una complejidad $O(n)$.

Recuerde que solo se puede emplear una pila auxiliar para esta labor. No debe mover los elementos a la segunda cola, solo verificar si es posible.

Al finalizar el laboratorio, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este laboratorio.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi
Rony Cueva
Erasmus Gomez

San Miguel, 25 de octubre del 2025